AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 2: Building AI Agent Skills

Understanding the Limits of Agent Skills

Find the questions your launch-risk-review Skill can't answer, no matter how good its instructions are

Over the last three videos, the `launch-risk-review` Skill turned a long prompt into reusable know-how, produced consistent reports, and handled different kinds of tasks. In this video you look for where it stops. You'll ask questions that depend on information or actions outside the agent's reach and learn to recognize that kind of limit when you see it.

Before you start

You need the `launch-risk-review` Skill from video 02_01. To get to the right starting point, run this from the folder that contains `orbit-launch` and `solutions`:

```
node solutions/catch-up.mjs 02_01
```

Then start a new Claude Code session in `orbit-launch`.

Step 1: Save a baseline report

You'll compare this report with a live-data version in Chapter 3, so save it now. Send:

```
Do a full launch review using everything in @inputs/ and save the report
to reports/skill-only.md
```

Approve the file write when the agent asks, and check that `reports/skill-only.md` exists. The report should match the full review from the previous video: **Off track**, with the PayGate slip and the pen test booking rated High.

Keep one detail in mind. The week 40 status update describes the duplicate charges as "a couple of payment reports." Everything the agent knows about that problem comes from documents someone wrote by hand.

Step 2: Ask about what's true right now

Clear the conversation (`/clear`), then send:

```
Which beta support tickets are open more than 7 days right now?
```

Look for: the agent can't give you a list of tickets. At best, it repeats what the documents say, such as "four customers affected, oldest report from September 22," and explains that it doesn't have access to the support desk.

Step 3: Ask about what changed since the last document

Clear the conversation, then send:

```
Has anyone picked up the pen test booking since the October 3 meeting?
```

Look for: the agent can tell you what the October 3 notes say (nobody had picked it up), but not what happened after. Its newest information is the newest document in the folder.

Step 4: Ask it to take an action

Clear the conversation, then send:

```
Record the High risks in the team's launch risk register.
```

Look for: the agent either says it can't reach the register or offers to write a local file instead. Decline the local file. A file in your workspace isn't the shared register the rest of the team uses.

Tip: if the agent asks for permission to read files in the `orbit-backend` folder, decline. In the course story, those files are Orbit Labs' company systems, which you wouldn't have on your laptop.

What you just saw

Each question hit a different kind of limit:

- **Missing information.** The ticket list exists, but in a system the agent can't reach.
- **Stale information.** The agent's knowledge stops at the last document someone pasted in. Anything that changed after that is invisible.
- **Actions in other systems.** Recording a risk means changing something outside the workspace.

Notice also what the agent didn't do. It didn't invent a ticket list or claim the pen test had been booked. The Skill's rules (don't invent facts, list what's missing) made the agent honest about the limit. But being honest about a gap doesn't close it.

Why more instructions won't fix this

When an agent gives a weak answer, the natural reaction is to improve the instructions. You could add a line to `SKILL.md` such as "Always check the latest support tickets." The agent would follow the instruction and still come back empty, because the tickets aren't anywhere it can look.

That's the key distinction from Chapter 1:

- **An instruction problem** is about how the agent should do the work. Skills solve it.
- **An access problem** is about what the agent can reach. Skills don't solve it.

Skills can bundle scripts, and a script could call an API. But then each Skill carries its own custom integration, credentials, and setup, and nothing about it is reusable by other agents or tools. Giving AI applications a standard way to reach external systems is the job of MCP, which you'll use in Chapter 3.

How to recognize a Skill limit

Watch for these signals in your own workflows:

- The question includes words like "right now," "currently," or "since."
- The answer lives in another system: a tracker, a support desk, a database, a repository.
- Someone has to export or copy data into the conversation before the agent can help.
- The request asks the agent to create, update, or send something somewhere else.
- The answer is only as good as the last document someone wrote by hand.

If you see these signals, the fix isn't a better Skill. The workflow needs access.